



CSE110: Java OOP (Section-08)

Instructor: Sadia Nur Amin

Semester Summer 2026

Project Report

Smart Parking System – Project Report

GitHub Repository

<https://github.com/CodeWithPritom/Smart-Parking-System>

Clone : <https://github.com/CodeWithPritom/Smart-Parking-System.git>

Submitted By

Student ID	Student Name
2025-2-60-330	Arup Bhowmik Pritom
2025-2-60-323	Md. Montasirul Bin Abid
2025-1-60-142	Mehrab Morshed Marjan

1. Project Description

The **Smart Parking System** is a Java-based application designed to automate the management of vehicle parking. It streamlines the process of slot allocation, ticket generation, and fee calculation. The system is built using core Object-Oriented Programming (OOP) principles to ensure scalability, modularity, and efficiency.

Problem Statement

In urban areas, manual parking management is often slow, error-prone, and inefficient. Drivers struggle to find available slots, and calculating fees manually based on duration can lead to disputes. This digital system solves these issues by tracking slots in real-time and providing automated billing.

Objectives

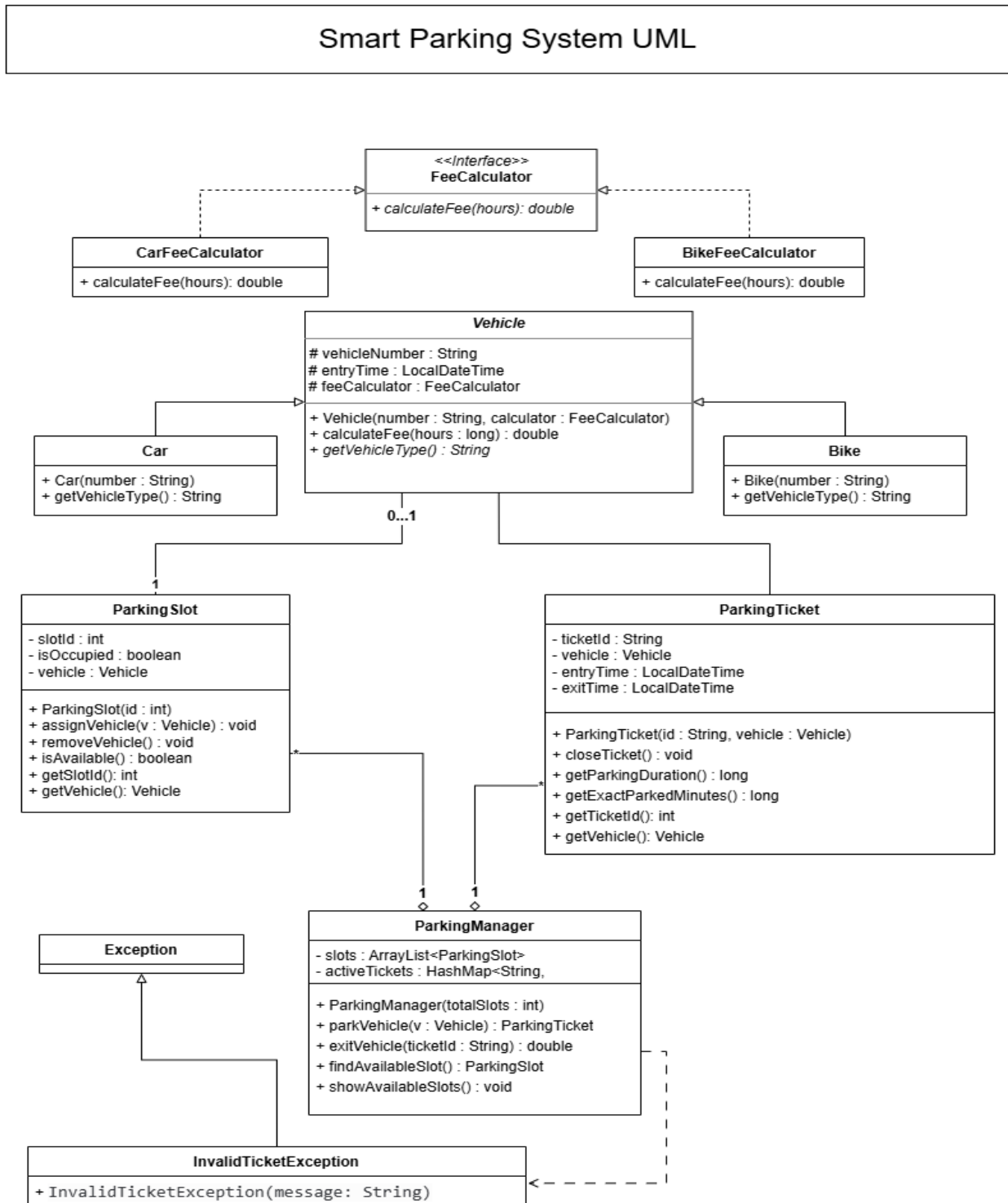
- To implement a structured management system for parking slots.
- To apply OOP concepts such as **Inheritance, Polymorphism, Abstraction, and Encapsulation**.
- To manage data efficiently using **Java Collections (ArrayList and HashMap)**.
- To handle runtime errors using robust **Exception Handling**.

2. Key Features

- **Vehicle Diversity:** Supports different vehicle types (Car and Bike) with specific fee structures.
- **Real-time Slot Tracking:** Automatically finds the next available slot and tracks occupancy.
- **Unique Ticket Generation:** Generates unique ticket IDs based on system timestamps.
- **Dynamic Fee Calculation:** Implements a **Strategy Pattern** to calculate fees based on vehicle type and duration.
- **Robust Input Handling:** Prevents system crashes through comprehensive exception management.

3. Program Design

3.1 UML



3.2 Program Structure

Main Components

- **Main (SmartParkingSystem):** Handles the command-line interface and user interaction.
- **ParkingManager:** The core controller that manages slots, tickets, and vehicle entry/exit.
- **Vehicle (Abstract Class):** The base class for all vehicle types.
- **Car & Bike (Subclasses):** Specific implementations of vehicles.
- **ParkingSlot:** Represents individual parking spaces.
- **ParkingTicket:** Manages entry/exit timestamps and duration logic.
- **FeeCalculator (Interface):** Defines the contract for fee calculation strategies.

Data Structures Used

- **ArrayList<ParkingSlot>:** Used to maintain an ordered list of parking slots for easy iteration and status checking.
- **HashMap<String, ParkingTicket>:** Used for O(1) constant-time lookup of active tickets using the Ticket ID.

4. Object-Oriented Concepts Used

4.1 Encapsulation

Data is protected by making class attributes private. Access is granted through public getter and setter methods.

- *Example:* ParkingSlot hides isOccupied and vehicle fields, ensuring they are only modified through assignVehicle() or removeVehicle().

4.2 Inheritance

The system uses a hierarchical structure where Car and Bike classes extend the abstract Vehicle class, inheriting common attributes like vehicleNumber and entryTime.

4.3 Abstraction

The Vehicle class is declared as abstract, and the FeeCalculator is an interface. This hides complex implementation details and only exposes necessary functionalities.

4.4 Polymorphism

- **Method Overriding:** Both Car and Bike override the getVehicleType() method.
- **Dynamic Method Dispatch:** The ParkingManager calls calculateFee() on a Vehicle reference, but the actual calculation is determined at runtime based on whether the object is a Car or a Bike.

5. Exception Handling

The system implements three types of exceptions to ensure stability:

1. **InputMismatchException:** Handles cases where users enter non-numeric values in the menu.
2. **InvalidTicketException (Custom):** A user-defined exception thrown when an exit is attempted with a non-existent or invalid Ticket ID.
3. **IllegalArgumentException:** Thrown if an attempt is made to register a vehicle without a valid license plate number.

6. Tools & Technologies Used

- Language: Java
- IDE: Apache NetBeans
- Libraries: Java Collections Framework

7. Role Assignment

- **Arup Bhowmik Pritom** → Core Logic, All Class Managing, Exception Handling, UML, Documentation, git&github Multi branch workflow Handling, Merge Conflict Handling, Code Deployment
- **Md. Montasirul Bin Abid** → Testing, Debugging,
- **Mehrab Morshed Marjan** → Parking System Implementation

9. Code

FeeCalculator Interface:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
 * license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package smart.parking.system;

/**
 *
 * @author User
 */
public interface FeeCalculator {

    /**
     *
     * @param hours
     * @return
     */
    double calculateFee( long hours);

}
```

BikeFeeCalculator Class:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
 * license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package smart.parking.system;
```

```

/**
 *
 * @author User
 */
public class BikeFeeCalculator implements FeeCalculator {
    @Override
    public double calculateFee(long hours){

        return hours*2000;
    }
}

```

CarFeeCalculator Class:

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
 * license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package smart.parking.system;

/**
 *
 * @author User
 */
public class CarFeeCalculator implements FeeCalculator {

    @Override
    public double calculateFee(long hours){

        return hours*2000; // Dhorì 2000 teka per hour :)
    }

}

```

Vehicle Class:

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
 * license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package smart.parking.system;

import java.time.LocalDateTime;

/**
 *
 * @author User
 */
public abstract class Vehicle {

    //Uml onusare variable nisi
    protected String vehicleNumber;
    protected LocalDateTime entryTime;
    protected FeeCalculator feeCalculator;
}

```

```

// constructor create korsi
// Inside Vehicle.java constructor
public Vehicle(String number, FeeCalculator calculator) {
    if (number == null || number.trim().isEmpty()) {
        throw new IllegalArgumentException("Vehicle number cannot be empty!");
    }
    vehicleNumber = number;
    entryTime = LocalDateTime.now();
    feeCalculator = calculator;
}

public double calculateFee(long hours) {

    return feeCalculator.calculateFee(hours); //calculateFee interface ore call dim ar
hours variable er modhe ja thakbe nibe
}

public abstract String getVehicleType(); // abstract dekhe sudhu body die rakhsi body car
and bike class ee banaitam
}

```

Bike Class:

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package smart.parking.system;

/**
 *
 * @author User
 */
public class Bike extends Vehicle{

    public Bike(String number){
        super(number, new BikeFeeCalculator());
    }

    @Override
    public String getVehicleType(){

        return "Bike";
    }

}

```

Car Class:

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template

```



```

    */
package smart.parking.system;

/**
 *
 * @author User
 */
public class Car extends Vehicle {

    public Car(String number){
        super(number, new CarFeeCalculator());
    }
    @Override
    public String getVehicleType(){

        return "Car";
    }
}

```

ParkingManager Class:

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
 license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package smart.parking.system;
//https://www.geeksforgeeks.org/java/java-util-package-java/

import java.util.*;
import java.lang.reflect.Array;
import java.util.ArrayList;
import java.util.HashMap;

/**
 *
 * @author User
 */
public class ParkingManager {

    private ArrayList<ParkingSlot> slots;
    private HashMap<String, ParkingTicket> activeTickets;
    public int totalSlots;

    public ParkingManager(int totalslots) {
        slots = new ArrayList<>();
        activeTickets = new HashMap<>();
        this.totalSlots = totalslots;

        for (int i = 1; i <= totalSlots; i++) {

            slots.add(new ParkingSlot(i));
        }

    }

    public ParkingSlot findAvailableSlot() {

```

```

        for (ParkingSlot Slot : slots) {

            if (Slot.isAvailable()) {
                return Slot;
            }

        }
        return null;
    }

    public ParkingTicket parkVehicle(Vehicle v) {

        ParkingSlot Slot = findAvailableSlot();

        if (Slot == null) {
            System.out.println("No Slot Available!");
            return null;
        }

        Slot.assignVehicle(v);

        String ticketId = "T" + System.currentTimeMillis();
        ParkingTicket ticket = new ParkingTicket(ticketId, v);

        activeTickets.put(ticketId, ticket);

        System.out.println("Vehicle Parked at slot: " + Slot.getSlotId());

        return ticket;
    }

    //throw new UnsupportedOperationException("Not supported yet."); // Generated from
    nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody
    public double exitVehicle(String ticketId) throws InvalidTicketException {
        ParkingTicket ticket = activeTickets.get(ticketId);

        if (ticket == null) {
            // Throw exception instead of just printing
            throw new InvalidTicketException("Ticket ID '" + ticketId + "' not found in the
system!");
        }

        ticket.closeTicket();

        long hours = ticket.getParkingDuration();
        double fee;
        fee = ticket.getVehicle().calculateFee(hours);

        for (ParkingSlot slot : slots) {
            if (slot.getVehicle() == ticket.getVehicle()) {
                slot.removeVehicle();
                break;
            }
        }

        activeTickets.remove(ticketId);

        long exactMinutes = ticket.getExactParkedMinutes();
        System.out.println("-----");
    }

```

```

        System.out.println("Total Parked Time: " + exactMinutes + " minutes");
        System.out.println("Billed Time: " + hours + " hour(s)");
        System.out.println("Total Parking Fee: " + fee + " Tk");
        System.out.println("-----");

        return fee;
    }

    public void showAvailableSlots() {
        // throw new UnsupportedOperationException("Not supported yet."); // Generated from
nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody
        for (ParkingSlot slot : slots) {
            if (slot.isAvailable()) {
                System.out.println("Slot " + slot.getSlotId() + " is free");
            }
        }
    }
}

```

ParkingSlot Class:

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package smart.parking.system;

/**
 *
 * @author User
 */
public class ParkingSlot {

    private int slotId;
    private boolean isOccupied;
    private Vehicle vehicle;
    String getSlotId;

    public ParkingSlot(int id) {
        slotId = id;
        isOccupied = false;
    }

    public void assignVehicle(Vehicle v) {

        vehicle = v;
        isOccupied = true;
    }

    public void removeVehicle() {
        vehicle = null;
        isOccupied = false;
    }

    public boolean isAvailable() {

```

```

        return !isOccupied;
    }

    public Vehicle getVehicle() {
        return vehicle;
    }

    public int getSlotId() {
        //throw new UnsupportedOperationException("Not supported yet."); // Generated from
nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody
        return slotId;
    }
}

```

ParkingTicket Class:

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package smart.parking.system;
//https://www.geeksforgeeks.org/java/java-time-duration-class-in-java/

import java.time.Duration;
import java.time.LocalDateTime;

/**
 *
 * @author User
 */
public class ParkingTicket {

    private String ticketId;
    private Vehicle vehicle;
    private LocalDateTime entryTime;
    private LocalDateTime exitTime;

    public ParkingTicket(String id, Vehicle vehicle) {
        ticketId = id;
        this.vehicle = vehicle;
        entryTime = LocalDateTime.now();
    }

    public void closeTicket() {
        exitTime = LocalDateTime.now();
    }

    //https://www.geeksforgeeks.org/java/java-time-duration-class-in-java/ ekhan theke paisi
    public long getParkingDuration() {
        long minutes = Duration.between(entryTime, exitTime).toMinutes();
        return (long) Math.ceil(minutes / 60.0); // Rounds up to 1 hour even for 5 mins
    }

    // Returning Minutes

```

```

    public long getExactParkedMinutes() {
        return Duration.between(entryTime, exitTime).toMinutes();
    }

    public String getTicketId() {
        return ticketId;
    }

    public Vehicle getVehicle() {
        //throw new UnsupportedOperationException("Not supported yet."); // Generated from
nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody
        return vehicle;
    }
}

```

InvalidTicketException Class:

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package smart.parking.system;

/**
 *
 * @author User
 */
public class InvalidTicketException extends Exception {
    public InvalidTicketException(String message) {
        super(message);
    }
}

```

SmartParkingSystem Class (Main):

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this template
 */
package smart.parking.system;

import java.util.InputMismatchException;
import java.util.Scanner;

/**
 *
 * @author User
 */
public class SmartParkingSystem {

    /**
     * @param args the command line arguments
     */
}

```

```
public static void main(String[] args) {
    // TODO code application logic here

    ParkingManager manager = new ParkingManager(5); // 5 Slots Passes

    Scanner sc = new Scanner(System.in);

    while (true) {

        System.out.println("\n1. Park Car");
        System.out.println("2. Park Bike");
        System.out.println("3. Exit Vehicle");
        System.out.println("4. Show Slots");
        System.out.println("5. Exit");

        int choice = 0;
        try {
            choice = sc.nextInt();
        } catch (InputMismatchException e) {
            System.out.println("Invalid input! Please enter a number between 1 and 5.");
            sc.nextLine(); // Clear the bad input
            continue;      // Restart the loop
        }

        if (choice == 1) {

            System.out.print("Enter Car Number: ");
            String num = sc.next();

            Vehicle car = new Car(num);

            ParkingTicket ticket = manager.parkVehicle(car);

            if (ticket != null) {
                System.out.println("Ticket ID: " + ticket.getTicketId());
            }

        } else if (choice == 2) {

            System.out.print("Enter Bike Number: ");
            String num = sc.next();

            Vehicle bike = new Bike(num);

            ParkingTicket ticket = manager.parkVehicle(bike);

            if (ticket != null) {
                System.out.println("Ticket ID: " + ticket.getTicketId());
            }

        } else if (choice == 3) {

            System.out.print("Enter Ticket ID: ");
            String id = sc.next();
            try {
                manager.exitVehicle(id);
            } catch (InvalidTicketException e) {
                System.out.println(e.getMessage());
            }

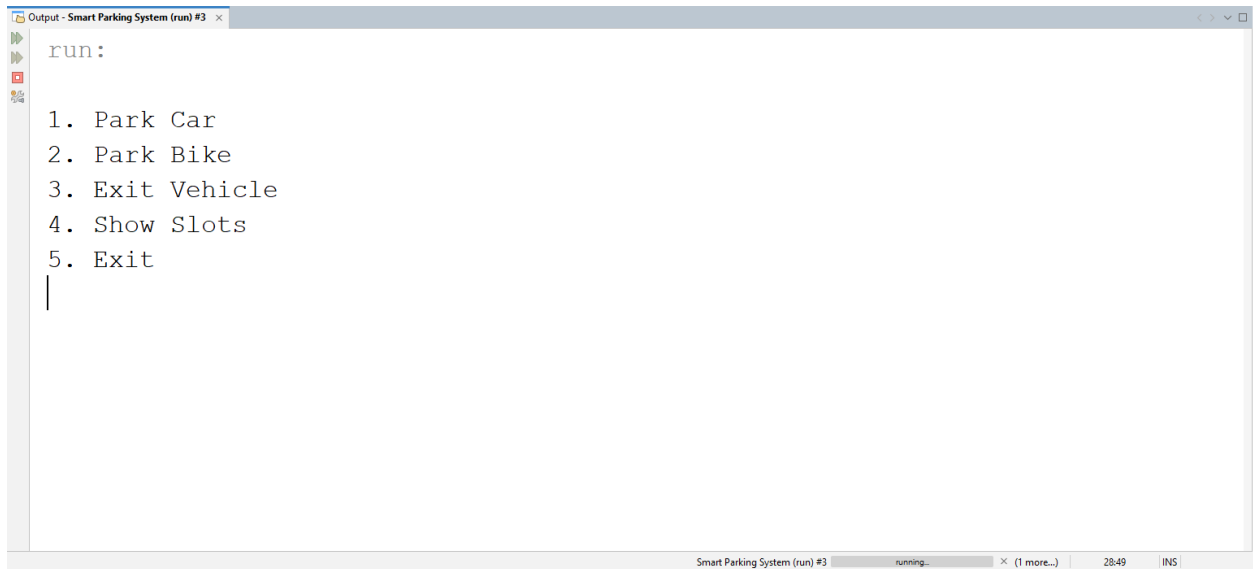
        }

    }

}
```

```
        }else if (choice == 4) {  
            manager.showAvailableSlots();  
        } else if (choice == 5) {  
            break;  
        }  
    }  
    sc.close ();  
}  
}
```

9. Sample Input/Output Screenshots



```
Output - Smart Parking System (run) #3 x
2. Park Bike
3. Exit Vehicle
4. Show Slots
5. Exit
1
Enter Car Number: Metro_4236
Vehicle Parked at slot: 1
Ticket ID: T1776738756506

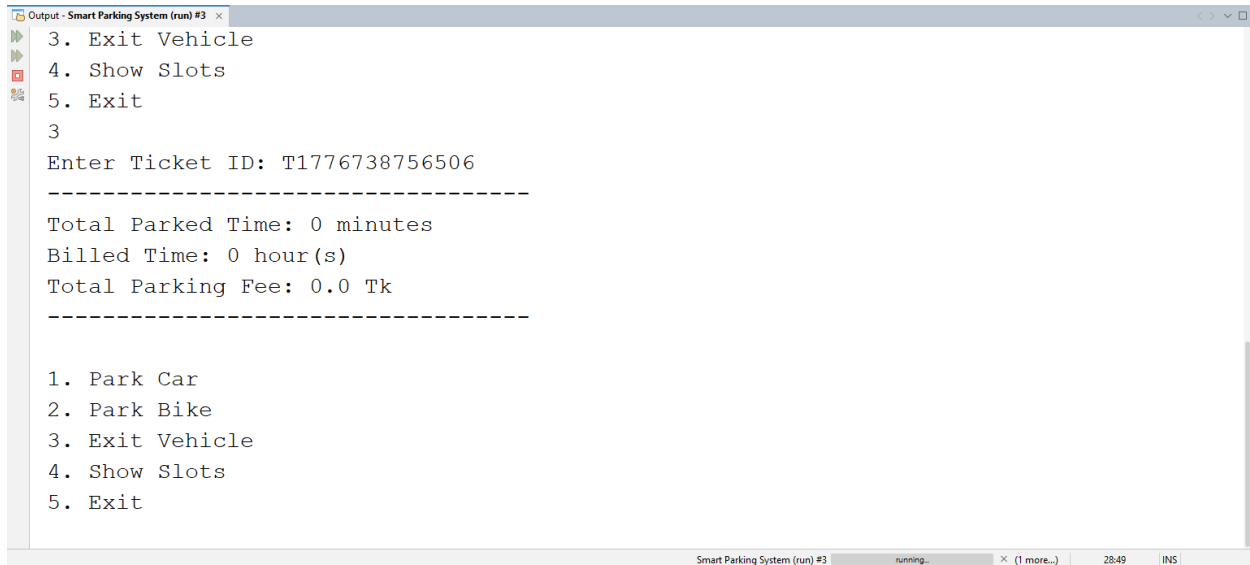
1. Park Car
2. Park Bike
3. Exit Vehicle
4. Show Slots
5. Exit
```

Smart Parking System (run) #3 x (1 more...) 28:49 INS

```
Output - Smart Parking System (run) #3 x
3. Exit Vehicle
4. Show Slots
5. Exit
4
Slot 2 is free
Slot 3 is free
Slot 4 is free
Slot 5 is free

1. Park Car
2. Park Bike
3. Exit Vehicle
4. Show Slots
5. Exit
|
```

Smart Parking System (run) #3 x (1 more...) 28:49 INS



```
Output - Smart Parking System (run) #3
3. Exit Vehicle
4. Show Slots
5. Exit
3
Enter Ticket ID: T1776738756506
-----
Total Parked Time: 0 minutes
Billed Time: 0 hour(s)
Total Parking Fee: 0.0 Tk
-----

1. Park Car
2. Park Bike
3. Exit Vehicle
4. Show Slots
5. Exit

Smart Parking System (run) #3    running...    x (1 more...)    28:49    INS
```

10. Challenges Faced

Challenge 1: Unique Ticket Id generation

Solution: We did not understand how can we generate unique ticket for people then we get an idea that we can use time in milli second to generate Random and unique Ticket Id because time is always changing in forward direction

```
String ticketId = "T" + System.currentTimeMillis();
```

Challenge 2: Navigate and store Parking Slot and Parking Ticket Problem

Solution : We were thinking that how can we create and manage parking slot and store and assign unique ticket according to parking slot. We used ChatGpt then find ArrayList and HashMap Libraries. This can temporally saved data . We could use file system but we do not want to use any storage system for this reason we used this libraries.

Reference :

1. <https://www.geeksforgeeks.org/java/java-util-hashmap-in-java-with-examples/>
2. <https://www.geeksforgeeks.org/java/arraylist-in-java/>

```
package smart.parking.system;
//https://www.geeksforgeeks.org/java/java-util-package-java/
```

```

import java.util.*;
import java.lang.reflect.Array;
import java.util.ArrayList;
import java.util.HashMap;

public class ParkingManager {
    private ArrayList<ParkingSlot> slots;
    private HashMap<String, ParkingTicket> activeTickets;
    public int totalSlots = 10;

    public ParkingManager(int totalslots){
        slots = new ArrayList<>();
        activeTickets = new HashMap<>();

        for(int i = 1; i <= totalSlots; i++){
            slots.add(new ParkingSlot(i));
        }
    }
}

```

Challenge 3: null problem !



```

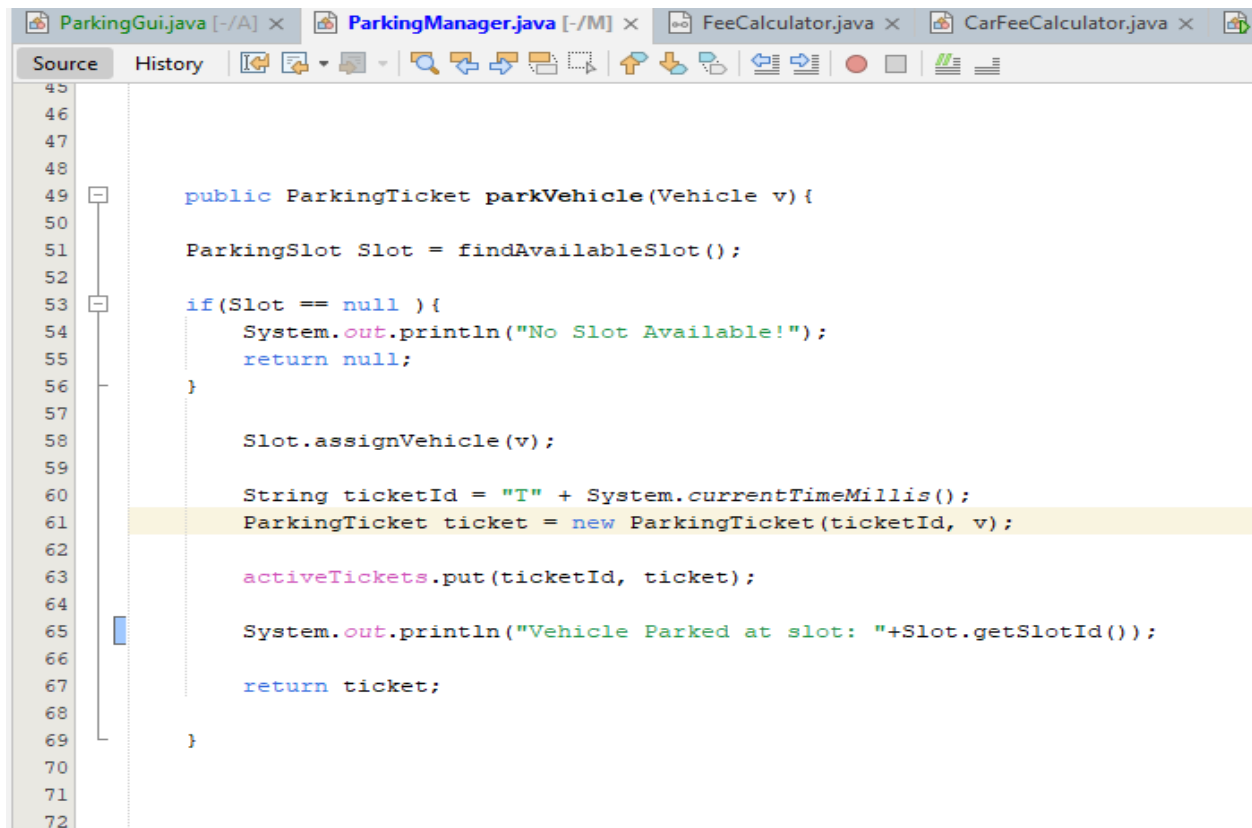
Output - Smart Parking System (run) #4 x
run:
1. Park Car
2. Park Bike
3. Exit Vehicle
4. Show Slots
5. Exit
1
Enter Car Number: 54325
Vehicle Parked at slot: null
Ticket ID: T1774533272193

```

Solution: We found the print text “Vehicle Parked at slot” then we saw that we did not put braces besides “getSlotId”

```
System.out.println("Vehicle Parked at slot: " + Slot.getSlotId);
```

Then we added braces then it is working properly now

A screenshot of an IDE window showing the source code of the `parkVehicle` method in `ParkingManager.java`. The window has multiple tabs: `ParkingGui.java`, `ParkingManager.java`, `FeeCalculator.java`, and `CarFeeCalculator.java`. The `Source` tab is active, showing the following code:

```
45  
46  
47  
48  
49 public ParkingTicket parkVehicle(Vehicle v) {  
50  
51     ParkingSlot Slot = findAvailableSlot();  
52  
53     if(Slot == null) {  
54         System.out.println("No Slot Available!");  
55         return null;  
56     }  
57  
58     Slot.assignVehicle(v);  
59  
60     String ticketId = "T" + System.currentTimeMillis();  
61     ParkingTicket ticket = new ParkingTicket(ticketId, v);  
62  
63     activeTickets.put(ticketId, ticket);  
64  
65     System.out.println("Vehicle Parked at slot: "+Slot.getSlotId());  
66  
67     return ticket;  
68  
69 }  
70  
71  
72
```

Challenge 4: Precise Billing Logic

Problem: Standard integer division for hours would result in \$0 fees for durations under 60 minutes.

Solution: We used `Math.ceil()` with Duration in minutes to ensure that any fraction of an hour is rounded up to the next full hour, following real-world parking policies.

```
public long getParkingDuration() {  
    long minutes = Duration.between(entryTime, exitTime).toMinutes();  
    return (long) Math.ceil(minutes / 60.0); // Rounds up to 1 hour even for 5 mins  
}
```

11. Conclusion

This project helped us understand real-world application of Object-Oriented Programming.

We learned:

- Designing modular systems
- Applying OOP concepts practically

- Managing data using collections
- Handling runtime errors

12. References

- Oracle Java Documentation (java.time and java.util packages)
- GeeksforGeeks (Java Collections Framework)
- W3Schools
- YouTube (Alex Lee, CodeWithHarry, Apna College)
- Course Lecture Notes on OOP Principles by Sadia Nur Amin.